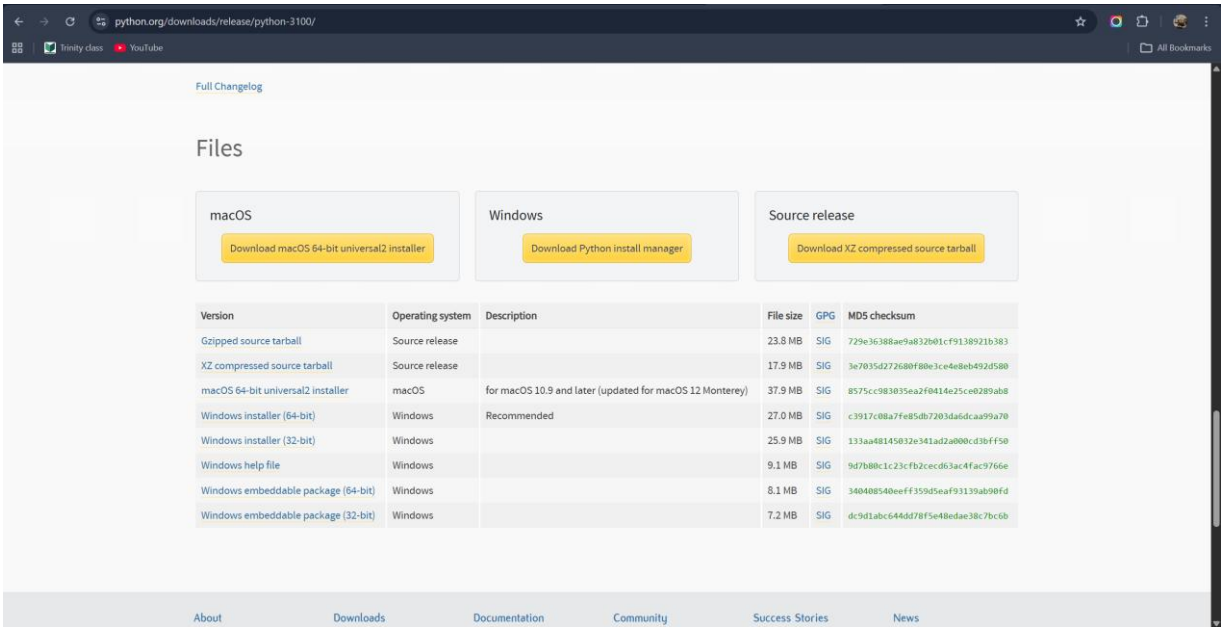
# Installation & User Guide

*Comparative Evaluation of Vision-Based Finger Tracking for Piano Interaction*

## 1. System Requirements

Before installing, confirm your system meets the following prerequisites:

- Python 3.10 (required — MediaPipe 0.10.x is incompatible with Python 3.11+)
- Windows 10 / 11 (primary platform; macOS and Linux work with minor path changes)
- Git (for cloning the repository)
- A USB webcam or built-in camera
- A MIDI keyboard connected via USB
- ~2 GB free disk space (model weights + Python packages)




*Python 3.10 download page at python.org/downloads*

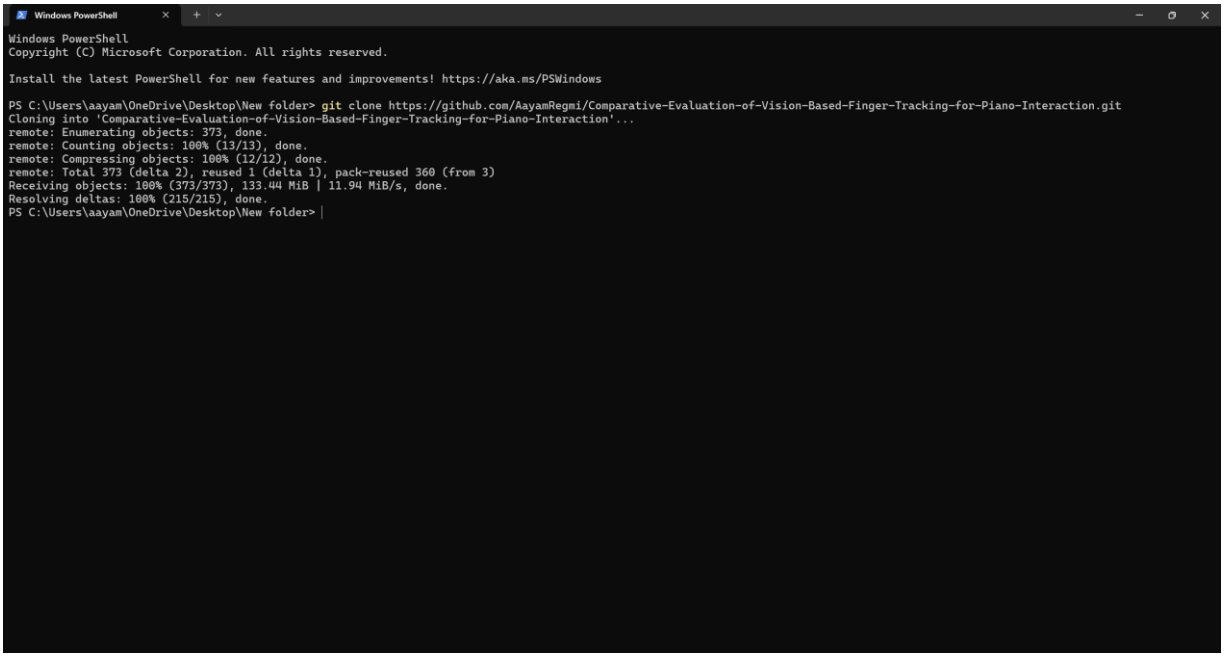
> **WARNING** Do not use Python 3.11 or higher. mediapipe==0.10.11 will fail to install or crash at runtime on anything above 3.10.

## 2. Step-by-Step Installation

### 2.1 Clone the Repository

Open a terminal and run:

```
git clone https://github.com/AayamRegmi/Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction.git
cd Comparative-Evaluation-of-Vision-Based-Finger-Tracking-for-Piano-Interaction
```

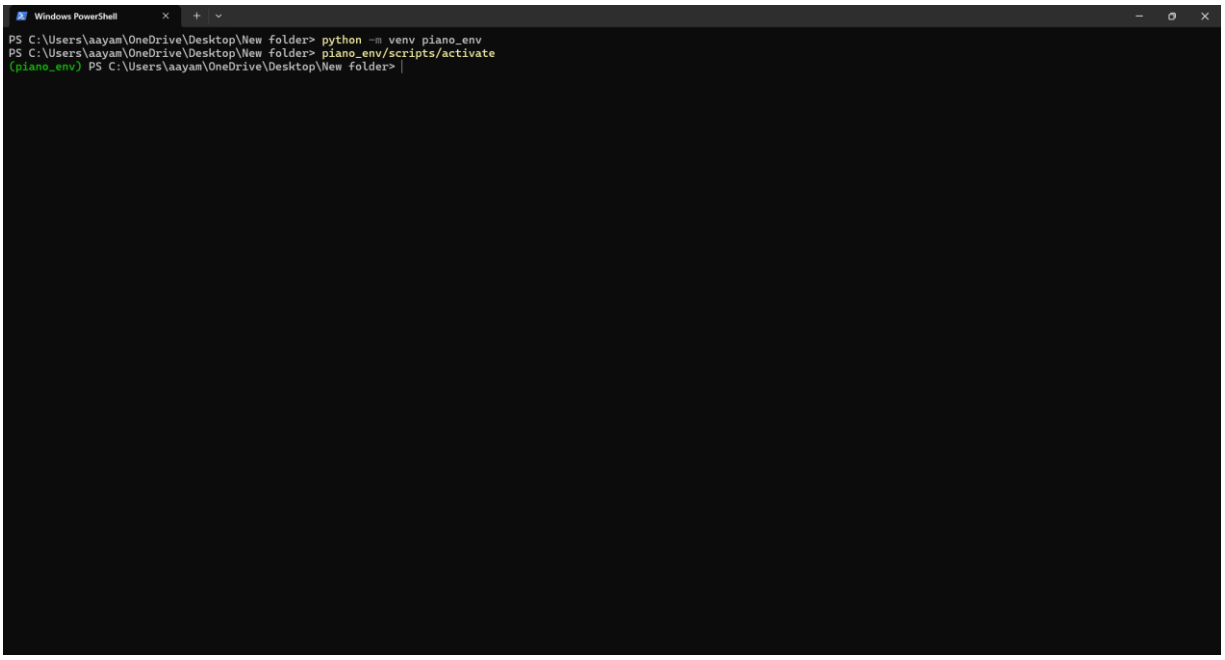



*Terminal window showing successful git clone output*

### 2.2 Create a Virtual Environment

Always use a dedicated virtual environment to avoid package conflicts:

```
python -m venv piano_env
```




*Terminal showing virtual environment creation (piano_env folder created)*

## 2.3 Activate the Virtual Environment

Windows (Command Prompt / PowerShell):

```
piano_env\Scripts\activate
```

macOS / Linux:

```
source piano_env/bin/activate
```

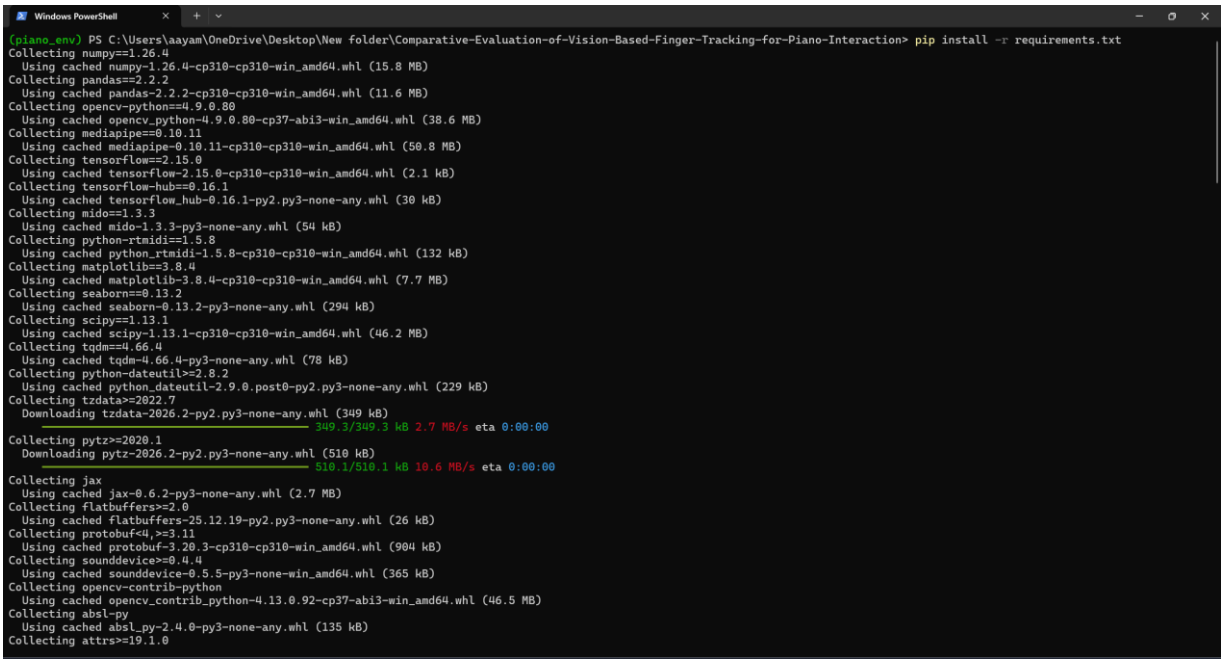
Your terminal prompt should show (piano_env) once activated.

## 2.4 Install Python Dependencies

With the virtual environment active, install all required packages from the requirements file:

```
pip install -r requirements.txt
```

This installs: NumPy, Pandas, OpenCV, MediaPipe, Mido, python-rtmidi, Matplotlib, Seaborn, SciPy, and tqdm.




*Terminal showing pip install completing with no errors*

> **WARNING** Windows only — python-rtmidi requires the Microsoft C++ Build Tools to compile. If the install fails with a compiler error, download and install 'Build Tools for Visual Studio' from visualstudio.microsoft.com (select Desktop development with C++), then re-run the pip command.

## 2.5 Install Additional Packages

Three packages used by analysis and results scripts are not included in requirements.txt and must be installed separately:

```
pip install Pillow python-pptx python-docx
```

| Package | Version tested | Used by |
|---|---|---|
| Pillow | 12.x | live_preview.py — results overlay rendering |
| python-pptx | 1.x | redesign_pptx.py — presentation export |
| python-docx | any | Documentation helper scripts |

## 2.6 OpenPose Model Files

OpenPose hand tracking runs through OpenCV's DNN module using pre-trained Caffe weights. The model files are already included in the repository under scripts/ — no separate OpenPose installation is needed:

- scripts/openpose_hand.prototxt — network architecture
- scripts/openpose_hand.caffemodel — pre-trained weights (~55 MB)

> **NOTE** If the caffemodel file is missing (e.g. after a shallow clone), run python download_openpose_model.py from the project root to re-download it.

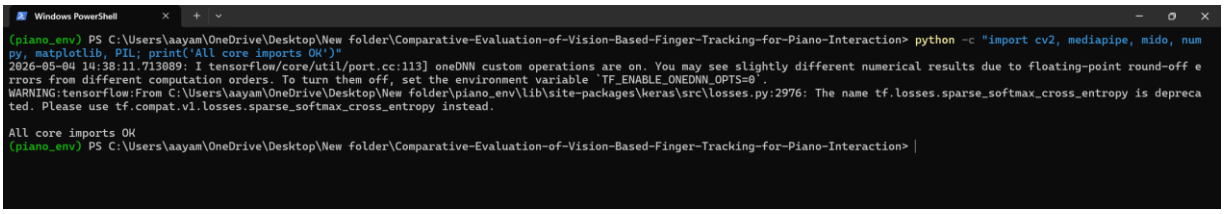
## 3. Verify the Installation

With the virtual environment active, run the following one-liner to confirm all core libraries import successfully:

```
python -c "import cv2, mediapipe, mido, numpy, matplotlib, PIL; print('All core imports OK')"
```

Expected output:

```
All core imports OK
```

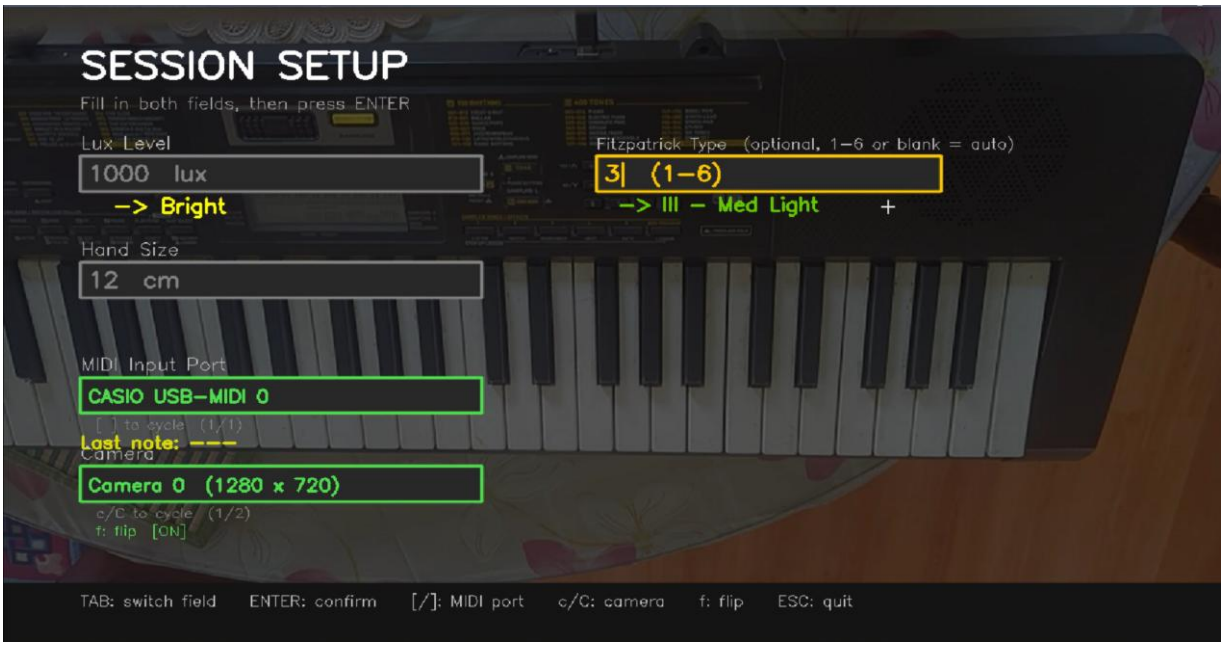



*Terminal showing 'All core imports OK' with no errors or warnings*

## 4. MIDI Device Setup

The system reads MIDI events in real time to log which keys are pressed during recording and testing sessions.

- Connect the MIDI keyboard to the computer via USB before launching any script.
- Windows may require a driver from the keyboard manufacturer — check the product page if the keyboard is not detected.
- On the setup screen that appears when launching record.py or live_preview.py, the detected MIDI ports are listed.
- Use the [ and ] keys to cycle between available ports and select the correct one.
- Press a key on the piano to confirm the 'Last note' field updates — this confirms the MIDI connection is working.




*Setup screen showing the MIDI port selection box and 'Last note' live readout*

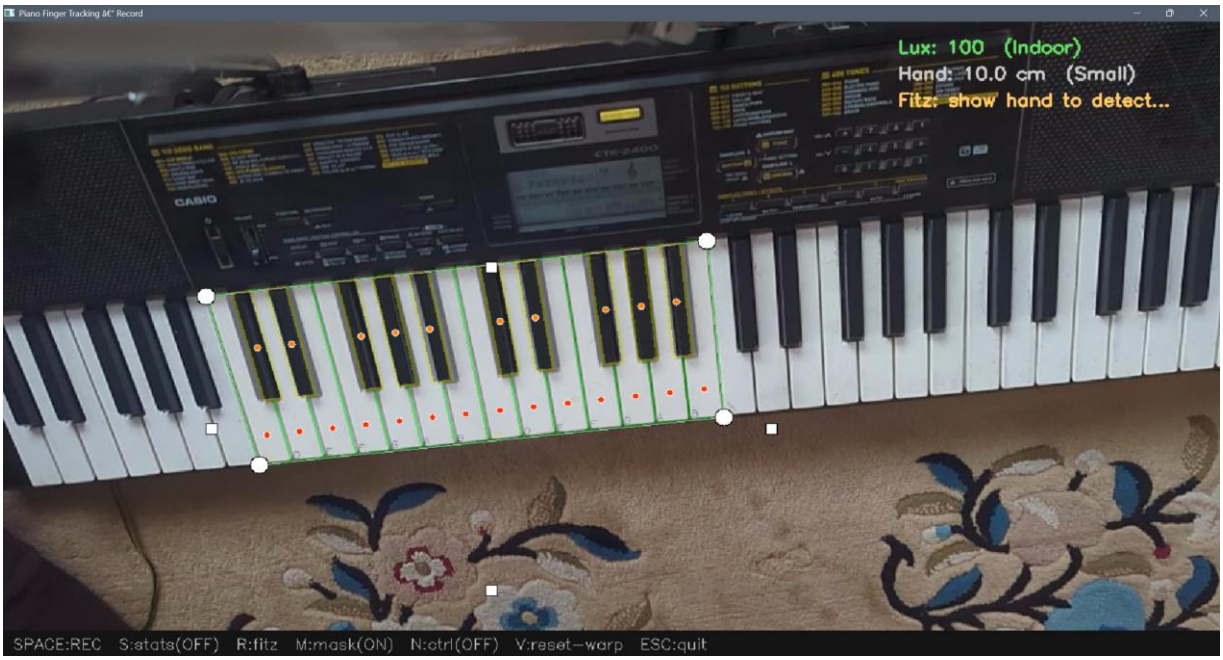
## 5. Running the Scripts

All scripts are run as Python modules from the project root with the virtual environment active. The table below summarises each script's purpose:

| Script | Command | Purpose |
|---|---|---|
| key_calibration | `python -m scripts.key_calibration` | Align the on-screen keyboard overlay to the camera view and save calibration |
| live_preview | `python -m scripts.live_preview` | Real-time MPJPE accuracy test — no data written to disk |
| record | `python -m scripts.record` | Record a labelled session: video, MIDI, and session metadata |
| analyse | `python -m scripts.analyse` | Offline MPJPE analysis of recorded sessions via GUI or CLI |
| plot_results | `python -m scripts.plot_results` | Generate all charts and participant reports from analysed data |

### 5.1 Key Calibration (key_calibration.py)

Run this first, or whenever the camera position changes. Drag the keyboard overlay to match the physical piano keys in the camera view, then press ENTER to save.



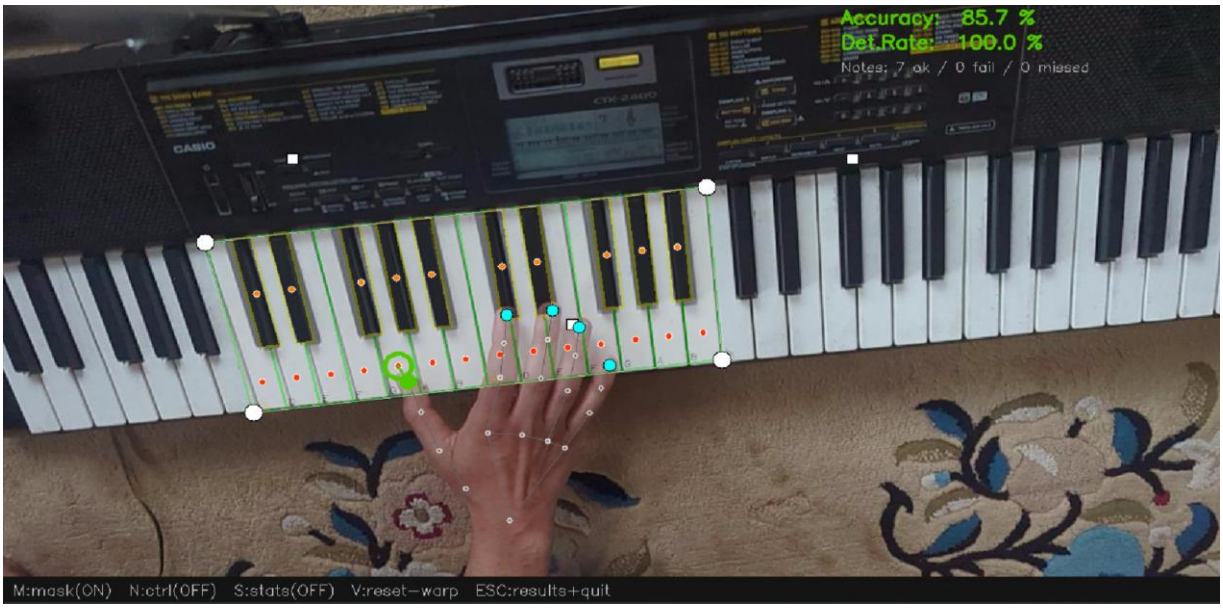

*key_calibration.py running — keyboard overlay aligned to physical piano keys*

Key controls:

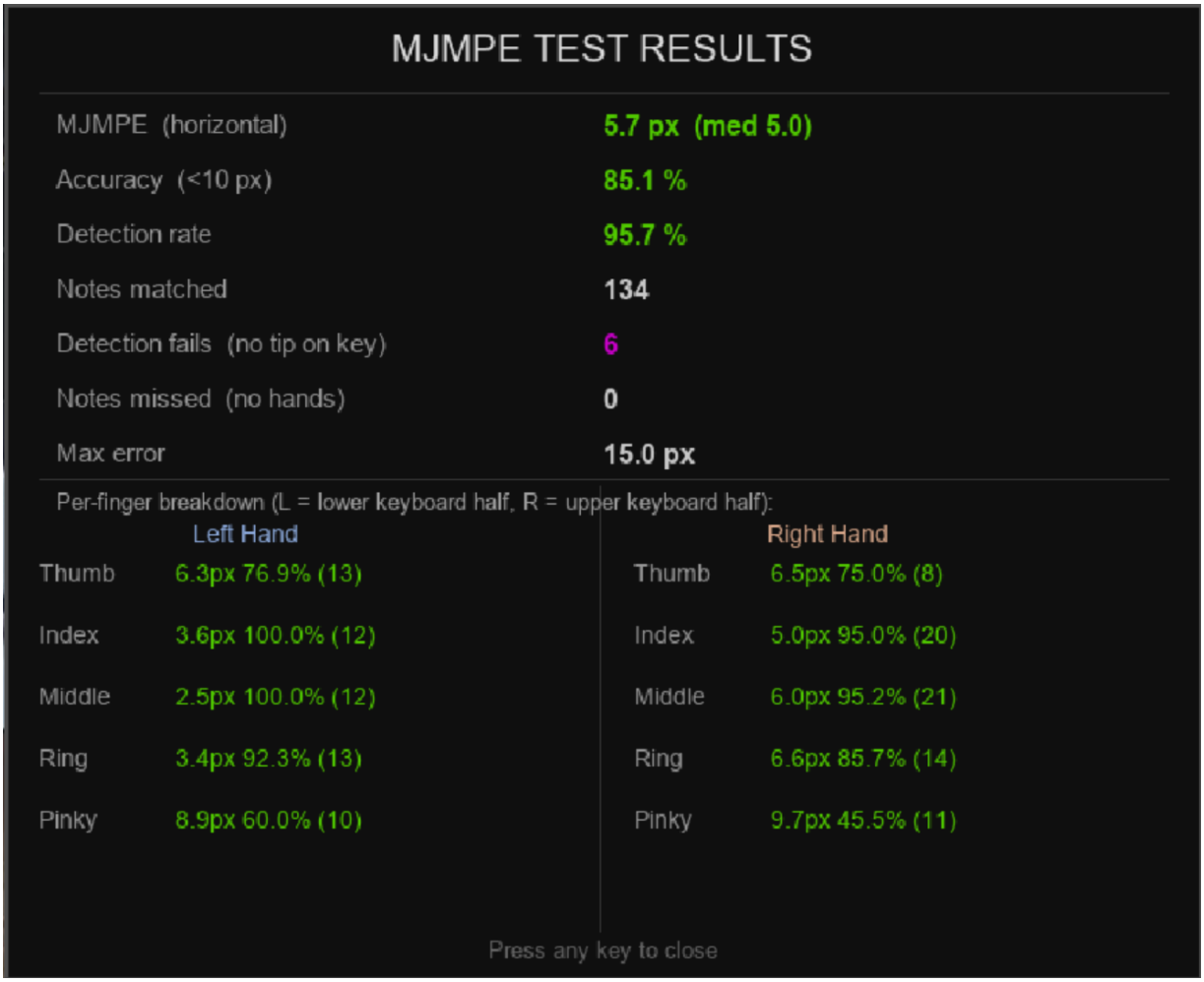
| Key | Action |
|---|---|
| Drag interior | `Move the whole overlay` |
| Drag edges | `Resize key width or height` |
| Drag corner ◯ | `Warp perspective to correct camera angle` |
| Scroll wheel | `Add or remove a key on the right` |
| [ / ] | `Shift the starting MIDI note` |
| C | `Toggle centre visualisation (line / dot)` |
| F | `Toggle 180° flip to match record.py` |
| H | `Toggle help overlay` |
| ENTER or S | `Save calibration` |
| ESC | `Quit without saving` |

## 5.2 Live MPJPE Test (live_preview.py)

Opens a setup screen to select the MIDI port and camera. Once running, play the piano and MPJPE accuracy is shown live in the top-right corner. Press ESC to stop and view the final results panel.




*live_preview.py main view — hand skeleton, key overlay, MPJPE readout top-right*



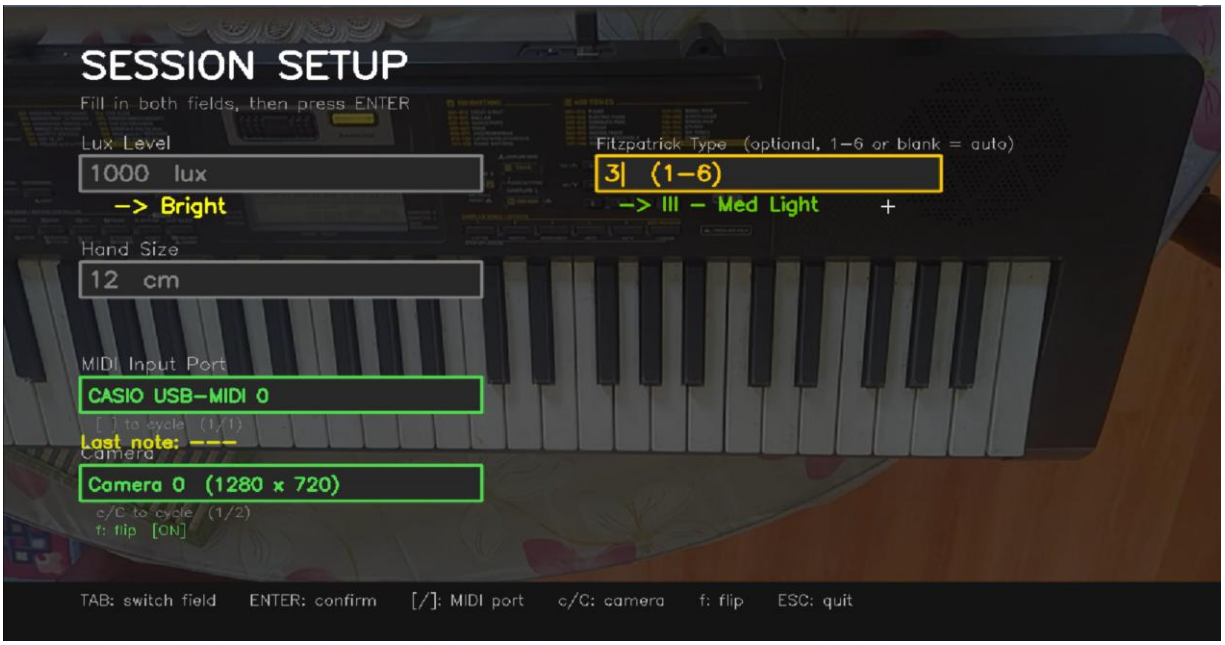

*live_preview.py results panel showing MPJPE, accuracy %, and per-finger breakdown*

Key controls during test:

| Key | Action |
|---|---|
| M | Toggle key mask overlay on/off |
| N | Toggle mask control panel |
| C | Toggle centre visualisation (line / dot) |
| S | Toggle FPS / latency stats overlay |
| V | Reset perspective warp to rectangle |
| ESC | Stop test and show results |

## 5.3 Recording Session (record.py)

Records a full participant session. The setup screen collects participant ID, Fitzpatrick skin type, ambient lux, and hand size before recording begins. Video, MIDI, and metadata are saved to data/raw/p###/.

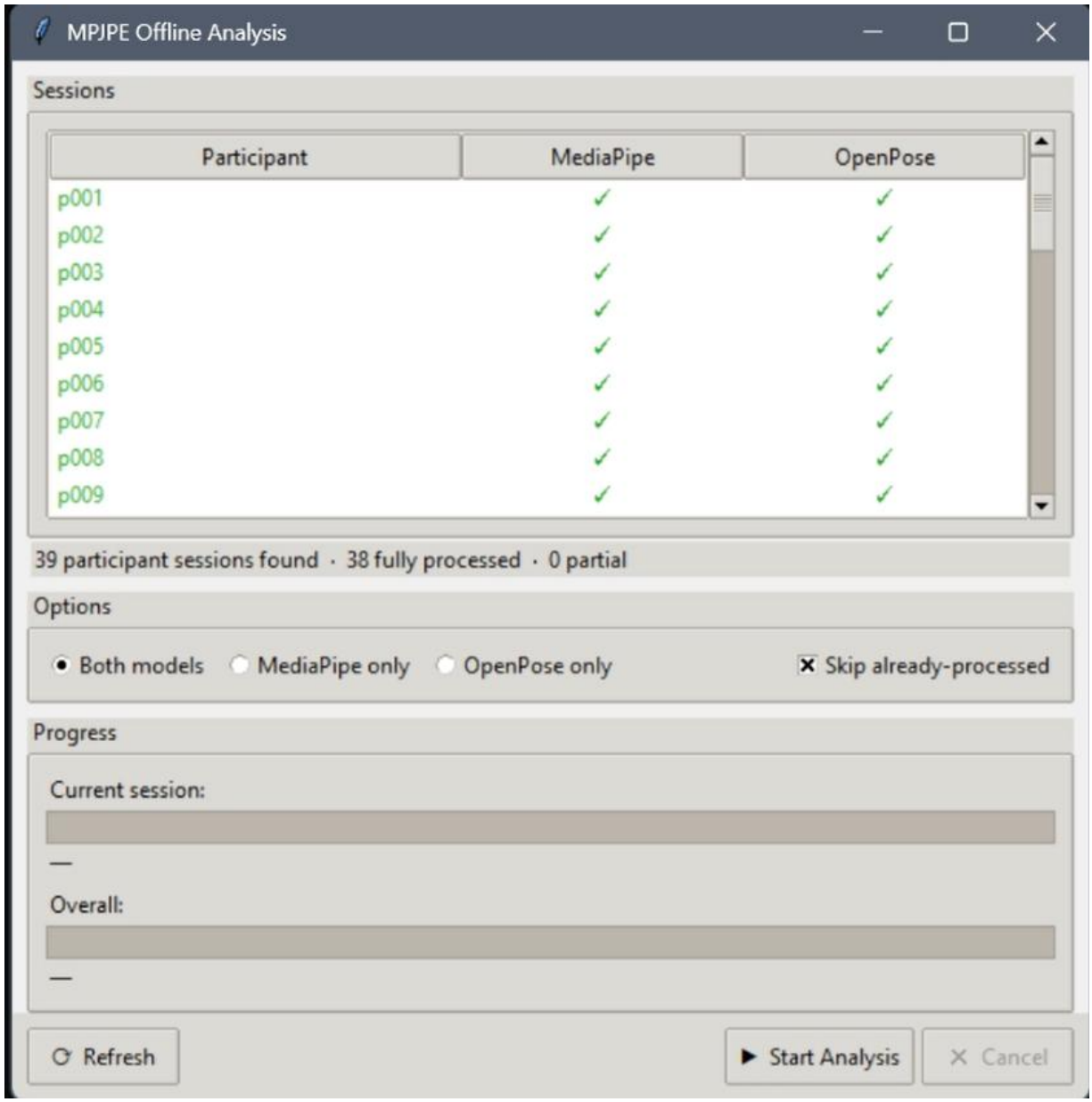



*record.py setup screen — participant ID, Fitzpatrick type, lux, and hand-size fields*

## 5.4 Offline Analysis (analyse.py)

Processes recorded sessions to compute MPJPE metrics for MediaPipe and/or OpenPose. Can be run via the GUI (double-click sessions to process) or from the command line:

```
python -m scripts.analyse                          # open GUI
python -m scripts.analyse data/raw/p001/           # process one session
python -m scripts.analyse data/raw/p001/ --model openpose
```




*analyse.py Tkinter GUI showing session list with MediaPipe and OpenPose columns*

## 5.5 Plot Results (plot_results.py)

Reads all results JSON files under data/processed/ and generates group-level charts and per-participant PDFs in data/plots/:

```
python -m scripts.plot_results
```


*data/plots/ folder — generated PNG charts visible in file browser*

## 6. Data Folder Structure

The following folder structure is created automatically as sessions are recorded and processed:

```
data/
├── calibration/
│   └── key_centers.json          ← saved by key_calibration.py
├── raw/
│   └── p001/
│       ├── p001.mp4              ← raw video (no overlays)
│       ├── p001_frames.csv       ← frame timestamps
│       ├── p001_midi.jsonl       ← MIDI events
│       ├── p001_midi.mid         ← standard MIDI file
│       └── p001_session.json     ← session metadata
├── processed/
│   └── p001/
│       ├── p001_mediapipe_results.json
│       └── p001_openpose_results.json
└── plots/
    ├── 01_model_comparison_mpjpe.png
    └── ...                        ← all group charts
```

## 7. Troubleshooting

| Problem | Likely cause | Fix |
|---|---|---|
| pip install fails on python-rtmidi | `Missing C++ compiler` | `Install Visual Studio Build Tools (Desktop development with C++)` |

| | | |
|---|---|---|
| ModuleNotFoundError: mediapipe | Wrong Python version or venv not active | Confirm python --version is 3.10.x and (piano_env) prefix is shown |
| No MIDI ports detected | Keyboard not connected or driver missing | Connect keyboard before launching; install manufacturer USB-MIDI driver |
| Camera shows black / wrong camera | Wrong camera index in config.py | Edit CAMERA_INDEX in scripts/config.py (try 0, 1, 2…) |
| caffemodel missing error | Large file not pulled from Git LFS | Run python download_openpose_model.py to re-download |
| All core imports OK but MediaPipe crashes | Python 3.11+ installed | Switch to Python 3.10 and recreate the virtual environment |